Fleet Simulation Engine Design

1. Overview

This project is a discrete time based simulator for a fleet of vessels that are moving between ports. An external "commander" sends a single JSON command per line, and the “engine” answers each one with exactly one JSON line that describes the world state.

2. What Problem Does This Solve?

This code should be thought of as an "engine" of sorts. There is someone giving the orders for what each ship should do, and the engine's job is to apply those orders and report the world state at each tick.

Therefore, if we want the engine to be a source of truth for someone else's decisions, it has to be:

1. Deterministic: The same command stream must always produce the same output, so the driver can replay or debug a run.
2. All or nothing per command: A rejected command must leave the world exactly as it was, otherwise the driver cannot reason about the state after an error.
3. Strict about the protocol: There should be exactly one line per command, in order, even for garbage input. The reason for this is that the driver reads one line per command, so a doubled or skipped response could throw it off entirely.
4. Faithful to the spec's rules: Wherever the spec defines a message or a rule, the engine uses it word for word, so the output matches the reference transcripts exactly.

3. Input and Output

Each line of input is a command, which is a JSON object with a command name and a parameters object, like this one:

{"command": "set_destination", "parameters": {"vessel_id": "v1", "destination": 1}}

Commands are one per line because a line is a natural message boundary for a pipe, so the driver can write a command and read exactly one line back with no framing.

Blank lines are skipped instead of rejected, since they aren't commands and answering them would throw every later response off by one.

Every command gets exactly one line back, in one of two shapes:

- {"status": "ok", "payload": <world state>}
- {"status": "error", "payload": {"message": "vessel v1 is not idle"}}

Even invalid JSON or an internal bug produces exactly one error line, because the driver reads one response per command and a missing or extra line would match every later response to the wrong command.

The status comes first so the driver can branch on success or failure without reading the payload.

The engine flushes after every line, since Python buffers piped output and the driver would otherwise hang waiting.

Every ok response carries the whole world, so each response is self contained and the driver never has to rebuild the state from earlier messages.

4. State Model

The engine holds two versions of the world. Understanding why, and which one each check looks at explains almost every behavior in the codebase.

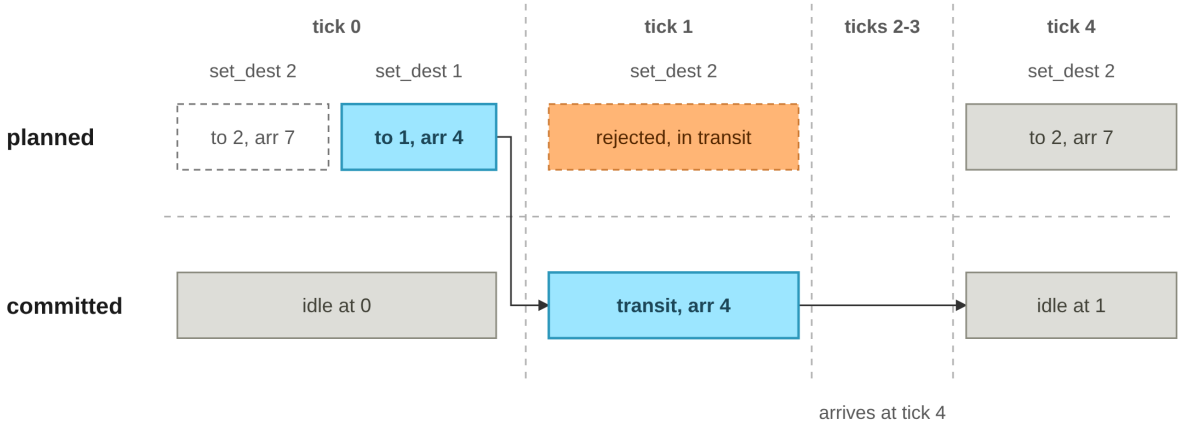
The committed state is the world as it actually is at the current tick, and it only changes when tick runs. In the code it isn't a separate field. committed is a property that returns history[-1], which is the newest entry in the history list.

The planned state is the committed state plus the commands sent so far this tick, and nothing more. It starts each tick as a deep copy of committed, and every accepted command edits it. Every ok response returns the planned state (except get_state_at, which returns a committed snapshot from history), so the driver always sees the effect of its last command right away. The next committed state is the plan plus three things tick adds: the tick number goes up by 1, arrivals happen, and new manifest cargo is added. So a vessel due to arrive next tick still shows transit in the plan, and the plan still shows the current tick number.

tick appends the planned state to history. That one append is what "committing" means: the plan becomes the newest history entry, so it becomes the committed state. Then tick makes a fresh deep copy for the next round of commands.

The figure below walks through the spec's base example for vessel v1, where going from port 0 to 1 takes 4 ticks and 0 to 2 takes 7. At tick 0 the driver sends set_destination 2 and then changes its mind to

1, so only the second one gets committed. At tick 1 another set_destination is rejected because the committed state says the ship is already in transit.



Every command checks the committed state and then writes only to the planned state. The spec words almost every command this way ("valid only if the committed state is idle", "valid if the committed state is docked"), but even without the spec, it turns out to be the right design for a few reasons:

1. You can change your mind before a tick: Since nothing is real until a tick, sending set_destination five times before it doesn't cause any problems because the last one is the one that actually gets committed when tick is updated.
2. Command order inside a tick wont matter for mode checks: Every command sees the same committed world, so load then undock and undock then load behave the same way. If the commands checked the plan instead, the outcome would depend on the order they were sent in.
3. Rejections are safe: Every command finishes all of its checks before its first write, so a rejected command never leaves half an update behind and prevents derailing the entire engine.
4. History doesn't need copying: Once a tick ends, nothing ever writes to its committed state again, so the engine can store those states directly and answer get_state_at instantly.

Why is committed a property instead of its own field? If committed were stored separately, it would have to be kept in sync with the newest history entry by hand, and any mistake would let the two drift apart. Getting it from history[-1] means there is only one source of ground truth of the current world.

There is one exception to all of this, which is that cargo amounts are checked against the planned state. The reason being if two loads in the same tick both checked committed inventory, they could both take the last unit on the dock. So the split ends up looking like this:

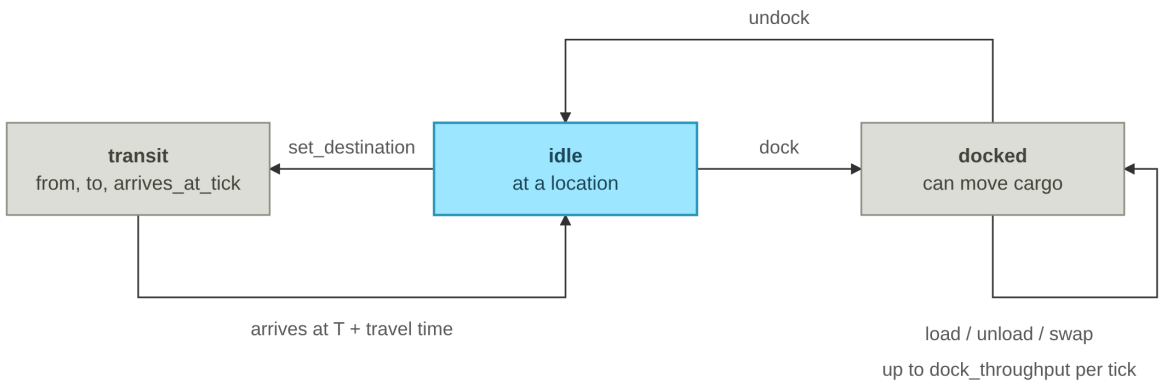
- Is the vessel idle or docked? Which then reads committed, because the spec defines it that way
- Where is the vessel? Which reads committed, since that is its current location
- Is there cargo to load or unload? Which reads planned, because cargo moves with every command

- How many dock operations are left? Ends up reading neither. It uses a per-dock counter that isn't part of the world state.

This behavior comes from the overarching idea that ship modes describe where a ship is, so they stay frozen for the tick. Cargo counts stack within the tick, because each load or unload really does change what's left on the dock for the next command.

5. Decisions the Spec Left to Me

The spec sheet left a good amount of behaviors "to the implementer". A vessel is always in exactly one of three modes, and the diagram below shows which commands move it between them. Each command is only a plan until the next tick commits it, while arriving happens on its own once a tick reaches arrives_at_tick.



Here is how I resolved each open question:

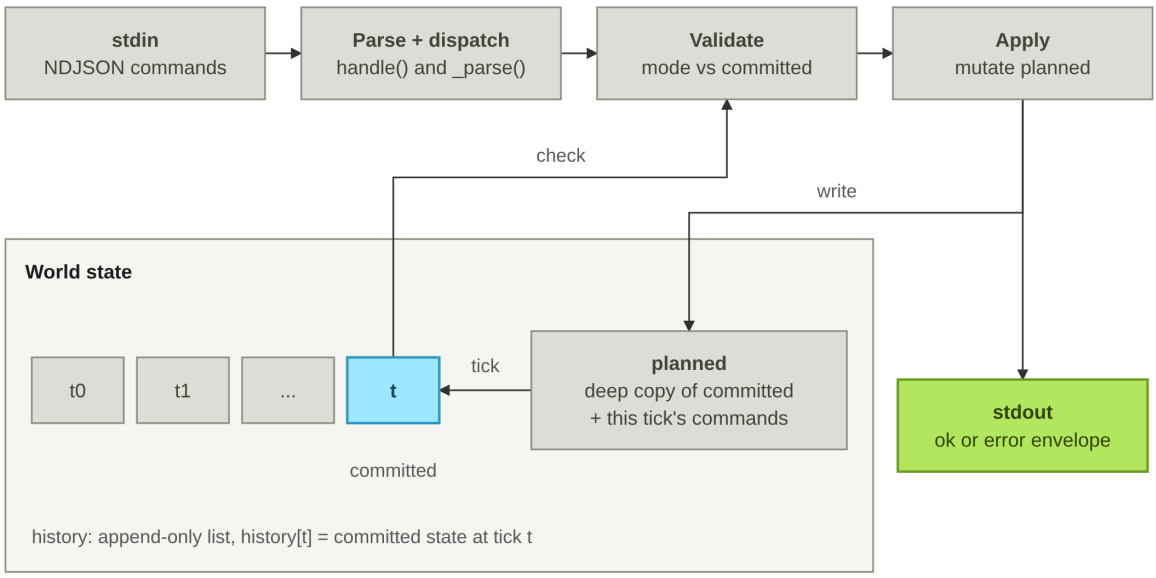
- set_destination to the current port is rejected. A zero tick trip isn't a real trip. arrives_at_tick would be the current tick, but nothing takes effect until the next tick, so the spec's rule that a ship arriving at tick T reports idle at T could never be met. In this code it would also leave the ship in transit forever, since tick only checks for the next tick.
- Docking twice, or undocking twice, in one tick is accepted and doesn't change anything. The committed mode still allows it, so the second command just sets the same plan again, which is admittedly redundant but doesn’t hamper functionality of the code.
- Undocking right after docking (or the other way around) in the same tick is rejected, because the committed mode doesn't allow it. The only way to change a planned dock is set_destination, which replaces it with a trip. There's no way to go back to plain idle within the same tick.
- Cargo moves and an undock in the same tick are both accepted, in either order. The vessel is still docked until the tick, so the cargo moves first and then it leaves with it.
- A load followed by an unload of the same unit counts as 2 operations. Throughput counts commands because the spec checks each command against the limit when it's issued.

- A swap counts as 1 operation, since the spec defines it as one and both halves are checked before either one moves.
- No dock_throughput in init means base mode. The engine removes the cargo fields from its output and rejects cargo commands.
- A malformed line gets exactly one error line and the engine keeps running, because the protocol promises one line per command.

Why not let undock cancel a dock from the same tick? That would make undock's validity depend on the plan, which goes against the committed/planned split. Keeping the split strict means every mode decision can be predicted from the committed state alone.

6. Architecture

The top row of the figure below is the path every line takes. It gets parsed, validated against the committed state, applied to the plan, and then exactly one line goes to stdout. The box underneath is the world state, which is the history list (remembering that the newest entry is the committed state) plus the plan that tick appends to it.

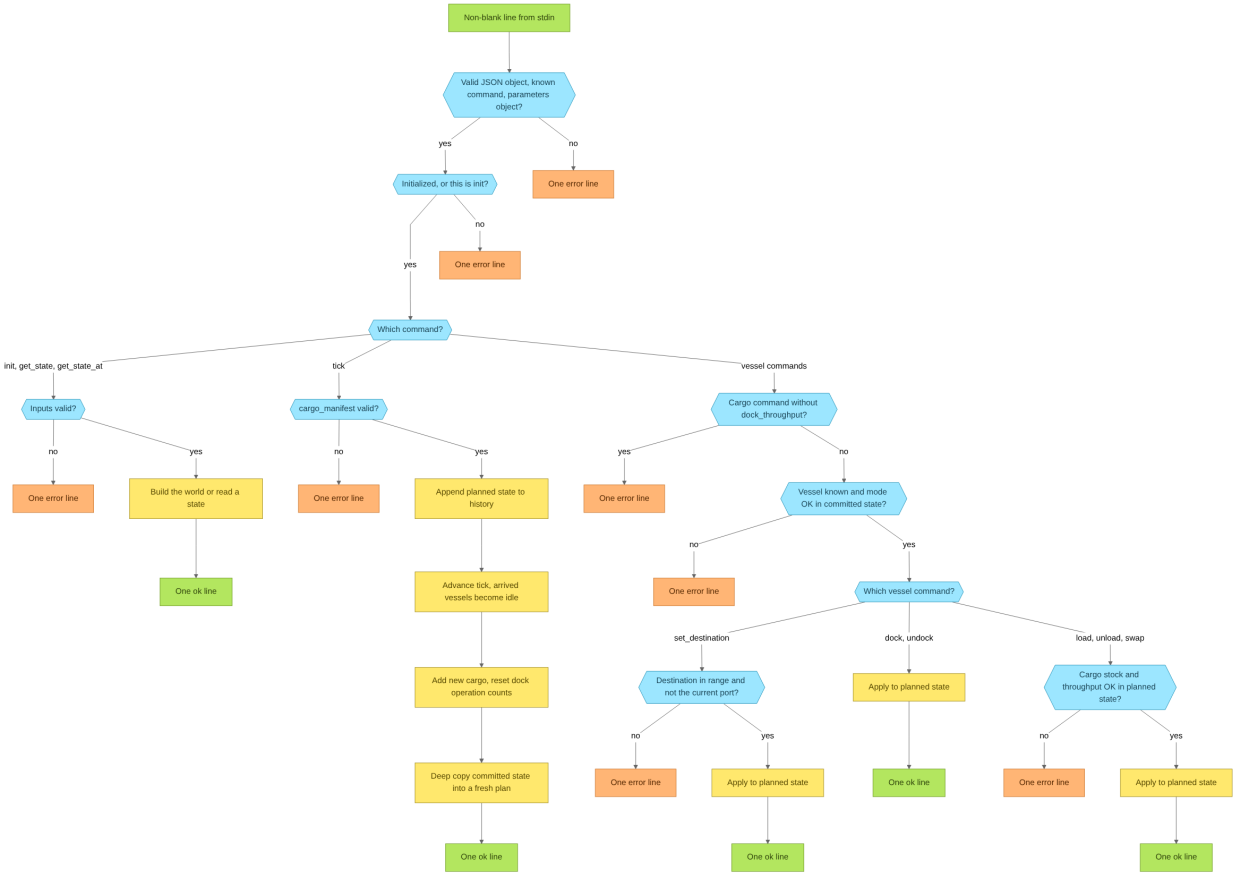


Everything lives in one Simulation object, and each part has one job:

- Parse + dispatch (handle, _parse): parses the JSON, rejects unknown commands and anything sent before init, calls the method with the same name, and puts the result in an ok or error envelope.
- Validate (_vessel, _check_*): mode checks read history[-1], which is the committed state. Cargo checks read the plan, and throughput uses a per-dock counter that isn't part of the world state.
- Apply (_plan, _move_*, _use_operation): only ever writes to planned and the dock counter.

- World state: history is append only, so history[t] is the committed state at tick t.
- tick: checks the cargo manifest and appends planned to history. Then, on that new entry, it adds 1 to the tick number, sets arrived ships to idle and adds new cargo. It resets the dock counts and makes a fresh copy for the next round last. Nothing touches that entry again after tick returns.

The flowchart below goes through this process visually. Notice that the outputs are just one lines in stdout.



7. Key Decisions and Tradeoffs

1. Including Two full snapshots - committed and planned
    - The reason for this is that each command just overwrites one vessel record in the plan, so "dock, then set_destination before the tick" is a plain overwrite with no conflict logic.
    - I decided to forgo a per-tick command log that gets replayed at tick, because it would need ordering and override rules for every pair of commands.
2. History stores committed states directly
    - The reason for this is that an entry never gets written after its tick ends, so there's no extra copy, and get_state_at is just an O(1) list lookup.

- I decided to forgo a separate history store with a deep copy every tick, which would double the memory for no real benefit.

3. Cargo as lists of {location, destination, quantity}
    - The reason for this is that lists are already the output format, and they keep insertion order stable, so the transcripts match line for line.
    - I decided to forgo dicts keyed by (location, destination). Lookups would be faster, but every response would need a conversion, and at this scale the lists are tiny anyway.
4. Throughput as a per-dock counter
    - The reason for this is that each load, unload or swap adds 1 before any cargo moves, and tick resets it. It's O(1), and it matches the spec's error text which was "transfer of 3 operations".
    - I decided to forgo working it out from the difference between committed and planned, which would hide a load followed by an unload.
5. Check everything before writing anything
    - The reason for this is that a rejected command leaves the plan untouched. Expected rejections raise CommandError, and anything unexpected becomes an internal error line instead of a crash.
    - I decided to forgo writing first and rolling back on failure, which is easy to get wrong, or letting exceptions crash the stream, which breaks the one line per command promise.

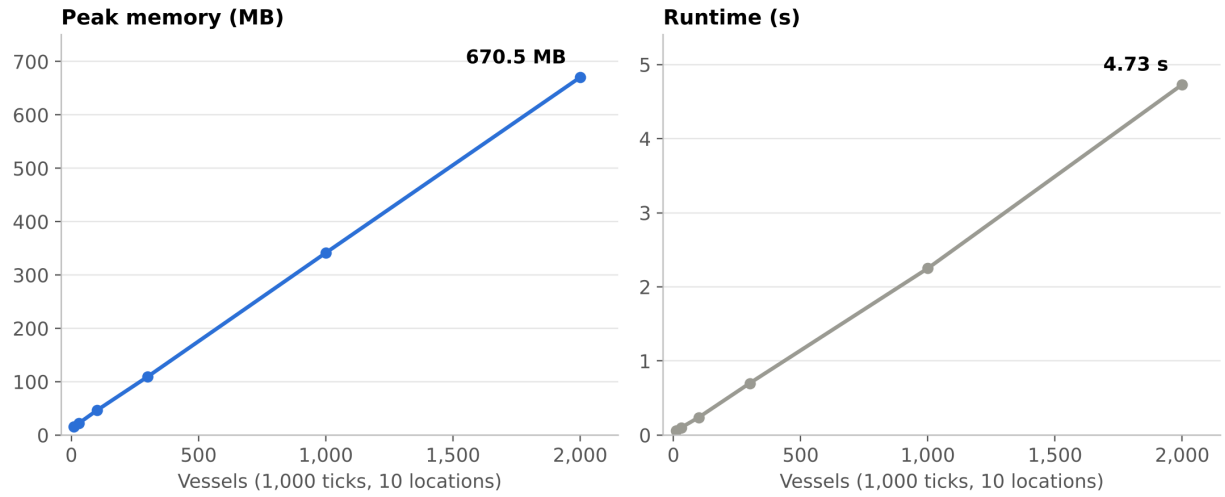
8. Testing and Evals

I tested this by running every reference transcript through sim.py and checking the output against the expected file. The reference transcripts cover the happy paths, so I focused on evalling the edge cases. A few scenarios that I have tested include: sending a ship to the port it's already at, docking then undocking in one tick, hitting the dock throughput limit, invalid JSON and commands before init.

9. Limitations

For long runs with idle ships, memory is the main limit. Every tick stores a full snapshot of the world, so memory grows with the number of vessels times the number of ticks.

The chart below shows peak memory and runtime over 1,000 ticks as the fleet grows from 10 to 2,000 vessels. Both of them grow in a straight line, at about 329 KB of memory per vessel for every 1,000 ticks.



That benchmark only uses idle ships, so it barely sends any commands. In a busier run where ships keep sailing and moving cargo, the output gets way bigger, in fact outpacing the memory growth. At 300 vessels it was about 3.6 GB of output, compared to 162 MB of memory. This is because every response prints the whole world, so in real use the response size would become a problem before history memory does.

The chart below gives an amazing depiction of the difference between the two.

